

Làm việc với

Tableau Public và MS SQL server

1. Export [sang CSV file](#) các dữ liệu từ MS SQL server, dùng Python script
 2. Export [sang Hyper file](#) các dữ liệu từ MS SQL server, dùng Python script
(Tableau Public 2024 đã không còn hỗ trợ trực tiếp)
 3. Sử dụng Web Data Connector
 - a. [Cách kiểm tra ODBC driver for SQL server](#)
 - b. Microsoft: [Download ODBC Driver for SQL Server](#).
 - c. [Cách thiết lập "Incremental Refresh"](#) (làm mới tăng trưởng) trong Tableau để chỉ tải thêm dữ liệu mới thay vì tải lại toàn bộ bảng
 - d. [Cách kiểm tra Tableau đã thực sự chạy Incremental](#) hay vẫn chạy Full Refresh
 4. Sử dụng các sản phẩm khác, thông qua trung gian (Google Sheets, Google Drive)
-

Tableau Public connect to MS SQL server, sử dụng Web Data Connector – WDC

Mục tiêu

Kết nối Tableau Public với dữ liệu lưu trữ trong database của MS SQL server thông qua Web Data Connector – WDC (tạm gọi là API)

Hạn chế

Tạo các WDC (API) để chuyển Dữ liệu giữa Tableau Public và database MS SQL server

Có thể cập nhật dữ liệu mới

Cách kiểm tra ODBC driver for SQL server

Microsoft: [Download ODBC Driver for SQL Server](#).

Cách thiết lập "Incremental Refresh" (làm mới tăng trưởng) trong Tableau để chỉ tải thêm dữ liệu mới thay vì tải lại toàn bộ bảng

Cách kiểm tra Tableau đã thực sự chạy Incremental hay vẫn chạy Full Refresh

Để hiện thực hóa việc kết nối **MS SQL Server** với **Tableau Desktop Public** thông qua **Web Data Connector (WDC)** ngay trên máy tính local, bạn cần xây dựng một "hệ sinh thái" nhỏ gồm 3 tầng.

Dưới đây là hướng dẫn chi tiết từng bước.

Cách tổ chức file

Bạn nên gom tất cả vào một thư mục dự án để dễ quản lý:

- \MyTableauProject
 - app.py (Backend Python)
 - index.html (WDC Frontend)
 - dashboard_v1.twbx (File Tableau của bạn)
-

Bước 1: Chuẩn bị môi trường (Local Setup)

Vì Tableau Public không thể "nhìn" thấy cơ sở dữ liệu trực tiếp, bạn phải biến dữ liệu SQL thành một dịch vụ web.

1. **Cài đặt Python:** Tải và cài đặt Python từ python.org.
2. **Cài đặt các thư viện cần thiết:** Mở Terminal/Command Prompt và chạy lệnh:

```
Bash
pip install flask flask-cors pyodbc
```

3. Đảm bảo SQL Server hỗ trợ kết nối:

- Mở **SQL Server Configuration Manager**.
- Đảm bảo **TCP/IP** trong phần *Protocols for MSSQLSERVER* đã được **Enabled**.

Bước 2: Tạo API trung gian (Python Flask), File Backend: **app.py** (Python)

Lớp này sẽ truy cập SQL Server và trả về dữ liệu định dạng JSON.

Tạo một file tên là **app.py**:

```
Python
from flask import Flask, jsonify
from flask_cors import CORS
import pyodbc

app = Flask(__name__)
CORS(app) # Rất quan trọng để Tableau không bị chặn bảo mật

def get_sql_connection():
    conn_str = (
        'DRIVER={ODBC Driver 17 for SQL Server};'
        'SERVER=localhost;' # Hoặc tên SQL Instance của bạn
        'DATABASE=Ten_Database_Cua_Ban;' # THAY THẾ TÊN DB
        'Trusted_Connection=yes;'
    )
    return pyodbc.connect(conn_str)

@app.route('/get-data', methods=['GET'])
def get_data():
    try:
        conn = get_sql_connection()
        cursor = conn.cursor()

        # THAY THẾ bằng câu lệnh SQL của bạn
        query = "SELECT TOP 100 * FROM Ten_Bang_Cua_Ban"
        cursor.execute(query)

        columns = [column[0] for column in cursor.description]
        results = [dict(zip(columns, row)) for row in cursor.fetchall()]

        conn.close()
        return jsonify(results)
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

```
if __name__ == '__main__':
    # Chạy tại port 5000
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Bước 3: Tạo Web Data Connector (WDC), File Frontend: **index.html**

Bạn cần 1 file HTML để Tableau Public "gọi" dữ liệu. Tạo file **index.html**:

```
HTML
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title> WDC - SQL Server and Tableau Public Edition</title>
    <script src="https://connectors.tableau.com/libs/tableauwdc-2.3.latest.js"></script>
    <script>
    (function() {
        var myConnector = tableau.makeConnector();

        // Định nghĩa cấu trúc bảng (Schema)
        myConnector.getSchema = function(schemaCallback) {
            var cols = [
                { id: "ID", alias: "ID", dataType: tableau.dataTypeEnum.int },
                { id: "Name", alias: "Tên", dataType: tableau.dataTypeEnum.string },
                { id: "Value", alias: "Giá trị", dataType: tableau.dataTypeEnum.float }
                // Lưu ý: Tên 'id' phải khớp chính xác với tên cột trong SQL
            ];

            var tableSchema = {
                id: "SQLServerData",
                alias: "Dữ liệu từ SQL Server Local",
                columns: cols
            };
            schemaCallback([tableSchema]);
        };

        // Lấy dữ liệu từ API Python
        myConnector.getData = function(table, doneCallback) {
            fetch('http://localhost:5000/get-data')
                .then(response => response.json())
                .then(data => {
                    table.appendRows(data);
                    doneCallback();
                });
        };

        tableau.registerConnector(myConnector);

        window.onload = function() {
            document.getElementById("submitButton").onclick = function() {
                tableau.connectionName = "SQL Data Source";
                tableau.submit();
            };
        };
    })();
    </script>
</head>

<body style="font-family: Arial; text-align: center; padding-top: 50px;">
    <h2>Kết nối SQL Server tới Tableau Public</h2>
    <button id="submitButton" style="padding: 15px 30px; cursor: pointer;">Tải Dữ Liệu</button>
</body>
</html>
```

Bước 4: Kết nối và Cập nhật dữ liệu

1. Khởi chạy hệ thống

1. Chạy file Python: `python app.py`.
API của bạn sẽ chạy tại `http://localhost:5000/get-data`.
Mở file `index.html` bằng một Web Server local
(Ví dụ: Dùng extension **Live Server** trong VS Code hoặc dùng lệnh `python -m http.server 8080` trong thư mục chứa file HTML).
2. Mở Tableau Desktop Public.
3. Chọn **Connect** -> **Web Data Connector**.
4. Nhập đường dẫn đến file `index.html` của bạn (Nếu bạn dùng VS Code, hãy dùng extension "Live Server" để có URL như `http://127.0.0.1:5500/index.html`).

2. Cách cập nhật khi dữ liệu thay đổi

Đây là phần quan trọng nhất vì Tableau Public có cơ chế làm mới khác với bản trả phí:

- **Trong Tableau Desktop Public (Local):** Khi dữ liệu trong SQL Server thay đổi, bạn chỉ cần quay lại tab **Data Source** trong Tableau và nhấn **Refresh**. Tableau sẽ gọi lại API Python và lấy dữ liệu mới nhất.
- **Trên Tableau Public (Web):** > ⚠ **Lưu ý quan trọng:** Tableau Public (Web) **không thể** tự động kết nối vào máy tính cá nhân (localhost) của bạn để lấy dữ liệu mới.
 - **Giải pháp:** Để cập nhật Dashboard đã đăng lên web, bạn phải mở file gốc trên máy tính, nhấn **Refresh** để cập nhật dữ liệu mới vào Extract, sau đó thực hiện **Save to Tableau Public** một lần nữa để ghi đè bản mới.

💡 Lời khuyên để quản lý hiệu quả

- **Dùng định dạng .twbx:** Luôn lưu file ở dạng Packaged Workbook để dữ liệu (Extract) được đóng gói kèm theo.
- **Kiểm tra Driver:** Đảm bảo máy bạn đã cài **ODBC Driver for SQL Server** (thường là bản 17 hoặc 18) để Python có thể kết nối thành công.

Cách kiểm tra ODBC driver for SQL server

Để đảm bảo mã Python (app.py) của bạn có thể kết nối thành công với SQL Server, việc kiểm tra chính xác tên và phiên bản của **ODBC Driver** là cực kỳ quan trọng. Chỉ cần sai một ký tự trong chuỗi kết nối (Connection String), hệ thống sẽ báo lỗi ngay.

Dưới đây là 3 cách để bạn kiểm tra trên máy tính Windows của mình:

Cách 1: Sử dụng ODBC Data Source Administrator

Đây là công cụ có sẵn trong Windows để quản lý các trình điều khiển kết nối dữ liệu.

1. Nhấn phím **Windows**, gõ từ khóa ODBC và chọn **ODBC Data Sources (64-bit)**. (Lưu ý: Luôn ưu tiên bản 64-bit vì Tableau và Python hiện nay đa số chạy trên nền tảng này).
2. Khi cửa sổ hiện ra, hãy chuyển sang tab **Drivers**.
3. Cuộn xuống dưới cùng để tìm các dòng có tên bắt đầu bằng **ODBC Driver... for SQL Server**.
4. **Ghi lại chính xác cái tên này.** * Ví dụ: ODBC Driver 17 for SQL Server hoặc ODBC Driver 18 for SQL Server.

Cách 2: Kiểm tra nhanh bằng PowerShell

Nếu bạn không muốn lội qua các menu, hãy dùng một dòng lệnh để liệt kê tất cả driver đang cài đặt.

1. Nhấn chuột phải vào nút Start, chọn **Windows PowerShell**.
2. Dán lệnh sau và nhấn Enter:

```
PowerShell
Get-OdbcDriver | Where-Object {$_.Name -like "*SQL Server*"}
```

3. Kết quả sẽ hiện ra cột **Name**. Đây chính là tên bạn cần điền vào biến conn_str trong file app.py.

Cách 3: Kiểm tra trực tiếp bằng Python (Khuyến dùng)

Cách này chắc chắn nhất vì nó cho bạn biết chính xác môi trường Python của bạn "nhìn" thấy những driver nào.

1. Tạo một file nhỏ tên là **check_driver.py** và dán đoạn code này vào:

```
Python
import pyodbc
drivers = [d for d in pyodbc.drivers() if 'SQL Server' in d]
print("Các Driver SQL Server khả dụng trên máy bạn là:")
for d in drivers:
    print(f"- {d}")
```

2. Chạy lệnh: **python check_driver.py**.

⚠ Lưu ý quan trọng về Driver 18

Nếu kết quả trả về là **ODBC Driver 18 for SQL Server**, bạn cần lưu ý một thay đổi nhỏ trong bảo mật của Microsoft. Driver 18 mặc định bật mã hóa (Encryption).

Nếu bạn gặp lỗi kết nối với Driver 18, hãy sửa chuỗi conn_str trong file app.py của bạn bằng cách thêm TrustServerCertificate=yes;;

```
Python
# Ví dụ cho Driver 18
conn_str = (
    'DRIVER={ODBC Driver 18 for SQL Server};'
    'SERVER=localhost;'
    'DATABASE=Ten_DB;'
    'Trusted_Connection=yes;'
    'TrustServerCertificate=yes;' # Thêm dòng này nếu dùng bản 18
)
```

Nếu bạn chưa thấy Driver nào?

Nếu danh sách trống rỗng, bạn cần tải và cài đặt bản chính thức từ Microsoft: [Download ODBC Driver for SQL Server](#).

Cách thiết lập "Incremental Refresh" (làm mới “tăng trưởng”) trong Tableau để chỉ tải thêm dữ liệu mới thay vì tải lại toàn bộ bảng

Việc thiết lập **Incremental Refresh** (Làm mới tăng trưởng) là một bước đi cực kỳ thông minh khi dữ liệu của bạn lớn dần theo thời gian. Thay vì bắt Tableau phải tải lại hàng triệu dòng dữ liệu cũ mỗi lần, nó sẽ chỉ "hỏi" SQL Server những dòng dữ liệu mới phát sinh.

Tuy nhiên, vì bạn đang dùng **Web Data Connector (WDC)**, quy trình này sẽ hơi khác so với kết nối trực tiếp. Bạn cần thực hiện thay đổi ở cả 3 nơi: **SQL Server, Python API và Tableau Desktop**.

1. Chuẩn bị tại SQL Server

Để làm mới tăng trưởng, bạn **bắt buộc** phải có một cột để xác định dữ liệu mới. Thông thường là:

- Một cột ID tự tăng (Identity/Primary Key).
- Hoặc một cột Thời gian (Created_At hoặc Updated_At).

2. Cập nhật mã nguồn (Cầu nối WDC)

Tableau sẽ gửi một giá trị gọi là `lastValue` (giá trị cuối cùng của lần cập nhật trước) đến WDC của bạn. Bạn cần sửa lại code để xử lý giá trị này.

Cập nhật `app.py` (Python):

Bạn cần sửa API để nhận thêm tham số lọc.

```
Python
@app.route('/get-data', methods=['GET'])
def get_data():
    # Lấy giá trị cuối cùng từ tham số URL (mặc định là 0 nếu không có)
    last_id = request.args.get('last_id', default=0, type=int)

    try:
        conn = get_sql_connection()
        cursor = conn.cursor()

        # Chỉ lấy những dòng có ID lớn hơn last_id
        query = f"SELECT * FROM Ten_Bang_Cua_Ban WHERE ID > {last_id} ORDER BY ID ASC"
        cursor.execute(query)

        columns = [column[0] for column in cursor.description]
        results = [dict(zip(columns, row)) for row in cursor.fetchall()]

        conn.close()
        return jsonify(results)
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Cập nhật `index.html` (JavaScript):

Trong hàm `getData`, bạn cần gửi `lastValue` sang Python.


```

JavaScript
myConnector.getData = function(table, doneCallback) {
    // Lấy giá trị cuối cùng mà Tableau đã lưu từ lần refresh trước
    var lastValue = table.incrementalValue || 0;

    var api_url = 'http://localhost:5000/get-data?last_id=' + lastValue;

    fetch(api_url)
        .then(response => response.json())
        .then(data => {
            table.appendRows(data);
            doneCallback();
        });
};

```

3. Thiết lập trong Tableau Desktop Public

Sau khi cập nhật code, bạn thực hiện các bước sau trong Tableau:

1. Tại tab **Data Source**, chuột phải vào nguồn dữ liệu chọn **Extract Data...**
2. Tích chọn **Incremental refresh**.
3. Tại mục **Identify new rows using column**, chọn cột tiêu chuẩn của bạn (ví dụ: ID).
4. Nhấn **Extract**.

4. Quy trình vận hành (Cực nhanh)

Từ bây giờ, mỗi khi bạn nhấn **Refresh**:

1. Tableau sẽ nhìn vào dữ liệu hiện có và thấy ID lớn nhất là **1000**.
2. Nó gửi lastValue = 1000 đến WDC của bạn.
3. API Python chạy lệnh: `SELECT * FROM ... WHERE ID > 1000`.
4. Nếu SQL Server có thêm 5 dòng mới (1001 đến 1005), chỉ có 5 dòng này được tải về và đắp thêm vào bộ nhớ Tableau.

Lưu ý quan trọng cho Tableau Public

Dù bạn dùng Incremental Refresh, giới hạn của Tableau Public vẫn tồn tại:

- **Offline Refresh:** Bạn chỉ có thể thực hiện việc này trên bản Desktop local.
- **Cập nhật trên Web:** Bạn vẫn phải nhấn nút **Save to Tableau Public** (ghi đè) sau khi làm mới để đẩy phần dữ liệu mới lên trang web. Tuy nhiên, thời gian làm mới trên máy bạn sẽ nhanh hơn gấp nhiều lần.

Kiểm tra xem Tableau đã thực sự chạy Incremental hay vẫn chạy Full Refresh

Để biết Tableau có đang thực sự "tiết kiệm sức lao động" cho máy tính bằng cách chạy Incremental hay vẫn "hùng hực" tải lại từ đầu (Full Refresh), chúng ta có 3 cách kiểm tra từ "ngoài vào trong" như sau:

1. Kiểm tra qua Nhật ký (Logs) của Flask

Đây là cách chính xác nhất vì bạn đang trực tiếp quan sát những gì Tableau yêu cầu từ phía máy chủ của mình.

- **Cách làm:** Nhìn vào cửa sổ Terminal/Command Prompt nơi bạn đang chạy file app.py.
- **Dấu hiệu:**
 - * **Full Refresh:** Bạn sẽ thấy dòng log yêu cầu tham số bằng 0 hoặc không có tham số: GET /get-data?last_id=0 HTTP/1.1
 - **Incremental Refresh:** Bạn sẽ thấy tham số last_id mang giá trị lớn nhất hiện có trong Tableau: GET /get-data?last_id=1250 HTTP/1.1 (Ví dụ 1250 là ID cuối cùng của lần cập nhật trước).

2. Thêm lệnh "Print" vào mã Python để xác nhận

Bạn có thể bắt mã Python tự "khai báo" tình trạng của nó mỗi khi có yêu cầu từ Tableau.

Cập nhật đoạn code trong app.py:

```
Python
@app.route('/get-data', methods=['GET'])
def get_data():
    last_id = request.args.get('last_id', default=0, type=int)

    # Dòng kiểm tra này sẽ hiện ra trong Terminal của bạn
    if last_id == 0:
        print(">>> ĐANG CHẠY FULL REFRESH (Lấy lại toàn bộ dữ liệu)")
    else:
        print(f">>> ĐANG CHẠY INCREMENTAL REFRESH (Chỉ lấy dữ liệu sau ID: {last_id})")

    # ... (phần code còn lại)
```

3. Kiểm tra trong giao diện Tableau Desktop

Tableau cung cấp một khu vực để bạn theo dõi lịch sử và trạng thái của các lần trích xuất dữ liệu.

1. Trong Tableau, nhấn vào menu **Data** ở phía trên cùng.
2. Chọn nguồn dữ liệu của bạn -> **Extract** -> **History...**
3. Một bảng sẽ hiện ra. Bạn hãy nhìn vào cột **Type**:

- **Full:** Nghĩa là Tableau đã xóa sạch dữ liệu cũ và tải lại mới hoàn toàn.
 - **Incremental:** Nghĩa là Tableau chỉ đắp thêm các dòng mới vào.
-

 Một lưu ý cực kỳ quan trọng

Để Tableau chạy Incremental, bạn không thể chỉ nhấn nút **Refresh** (F5) thông thường ở thanh công cụ. Bạn phải thực hiện đúng thao tác sau:

- Chuột phải vào nguồn dữ liệu trong tab **Data**.
- Chọn **Extract** -> **Append Data from Data Source** (nếu bạn muốn chạy thủ công).
- Hoặc khi bạn nhấn **Refresh** trong tab Data Source, hãy chắc chắn rằng cài đặt **Incremental Refresh** đã được lưu trong cấu hình Extract trước đó.

Nếu bạn thấy Tableau vẫn luôn gửi last_id=0, hãy kiểm tra lại file index.html xem dòng var lastValue = table.incrementalValue || 0; đã được viết đúng trong hàm getData chưa nhé.

Sử dụng Python để xuất dữ liệu ra định dạng **Text file CSV (.csv)**

Mục tiêu

Chuyển dữ liệu được lưu trữ trong database của MS SQL server sang dạng CSV file

Hạn chế

Dữ liệu trong CSV file, và database MS SQL server là độc lập nhau

1. Cài đặt thư viện cần thiết

Bạn cần cài đặt các thư viện sau để làm việc với SQL và Tableau Hyper:

```
pip install pyodbc pandas
```

2. Script Python (Dynamic SQL to CSV)

Script này sẽ thực hiện 3 yêu cầu của sau: Liệt kê bảng, cho phép chọn nhiều bảng, và xuất ra file .csv để Tableau load dữ liệu.

```
import pyodbc
import pandas as pd
import os

# --- CẤU HÌNH KẾT NỐI ---
# Nếu dùng Windows Authentication (Trusted_Connection), hãy giữ nguyên
# Nếu dùng tài khoản SQL, hãy thêm UID=username;PWD=password; vào chuỗi
conn_str = (
    'DRIVER={ODBC Driver 17 for SQL Server};'
    'SERVER=.....;' // Servername của bạn
    'DATABASE=.....;' // Databse name của bạn
    'Trusted_Connection=yes;'
)
OUTPUT_FOLDER="export_data"

def get_available_tables():
    """Liệt kê các bảng trong database"""
    try:
        conn = pyodbc.connect(conn_str)
        query = "SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE'"
        tables_df = pd.read_sql(query, conn)
        conn.close()
        return tables_df['TABLE_NAME'].tolist()
    except Exception as e:
        print(f"Lỗi kết nối: {e}")
        return []

def export_to_csv(selected_tables):
    """Chọn bảng và xuất dữ liệu ra CSV"""
    if not selected_tables:
        print("Không có bảng nào được chọn.")
```

```

        return

    conn = pyodbc.connect(conn_str)

    # Tạo thư mục 'exports' nếu chưa có để chứa file CSV
    if not os.path.exists(OUTPUT_FOLDER):
        os.makedirs(OUTPUT_FOLDER)

    for table in selected_tables:
        try:
            print(f"Đang trích xuất dữ liệu từ bảng: {table}...")
            query = f"SELECT * FROM {table}"
            df = pd.read_sql(query, conn)

            # Xuất file CSV (Sử dụng utf-8-sig để tránh lỗi font tiếng Việt nếu có)
            file_path = f"{OUTPUT_FOLDER}/{table}.csv"
            df.to_csv(file_path, index=False, encoding='utf-8-sig')
            print(f"-> Đã lưu file: {file_path}")
        except Exception as e:
            print(f"Lỗi khi trích xuất bảng {table}: {e}")

    conn.close()
    print("\nQuá trình hoàn tất!")

# --- LƯỖNG THỰC THI ---
if __name__ == "__main__":
    # 1. Liệt kê danh sách bảng
    all_tables = get_available_tables()

    if all_tables:
        print("\n=== DANH SÁCH BẢNG TRONG DATABASE ===")
        for i, table in enumerate(all_tables):
            print(f"{i+1}. {table}")

        # 2. Người dùng chọn bảng
        print("\nNhập số thứ tự để chọn bảng (cách nhau bằng dấu phẩy), 'all' để chọn tất cả:")
        user_input = input("Lựa chọn của bạn: ").strip().lower()

        selected_list = []
        if user_input == 'all':
            selected_list = all_tables
        else:
            try:
                indices = [int(x.strip()) - 1 for x in user_input.split(',')]
                selected_list = [all_tables[i] for i in indices if 0 <= i < len(all_tables)]
            except ValueError:
                print("Lỗi: Vui lòng chỉ nhập số hoặc 'all'.")

        # 3. Tiến hành xuất dữ liệu
        if selected_list:
            print(f"\nCác bảng đã chọn: {selected_list}")
            export_to_csv(selected_list)
        else:
            print("Không tìm thấy bảng nào hoặc lỗi kết nối.")

```

3. Quy trình vận hành và kết nối dữ liệu với Tableau

1. Chạy script Python để tạo folder **“export_data”**.
Folder này sẽ lưu các file CSV, tương ứng với từng table của database
2. Mở **Tableau Public**.
3. Tại màn hình Connect, chọn dưới mục **To a File** và chọn file .csv vừa tạo.

Sử dụng Python để xuất dữ liệu ra định dạng **Tableau Hyper (.hyper)**

Mục tiêu

Chuyển dữ liệu được lưu trữ trong database của MS SQL server sang dạng Hyper file

Hạn chế

Dữ liệu trong CSV file, và database MS SQL server là độc lập nhau

Có thể run Python script nhiều lần để lấy dữ liệu mới vào Hyper file và refresh Tableau

1. Cài đặt thư viện cần thiết

Bạn cần cài đặt các thư viện sau để làm việc với SQL và Tableau Hyper:

```
pip install pyodbc pandas pantab tableauhyperapi
```

2. Script Python (Dynamic SQL to Hyper)

Script này sẽ thực hiện 3 yêu cầu của sau: Liệt kê bảng, cho phép chọn nhiều bảng, và xuất ra file .hyper để Tableau load dữ liệu.

```
import pyodbc
import pandas as pd
import pantab
from tableauhyperapi import TableName

# --- BIẾN TOÀN CỤC ---
# Bạn có thể dễ dàng thay đổi tên file tại đây một lần duy nhất
OUTPUT_FILE = " DuLieu_Tu_MSSQL.hyper "

# Cấu hình kết nối MS SQL Server
conn_str = (
    'DRIVER={ODBC Driver 17 for SQL Server};'
    'SERVER=.....;' // Servername của bạn
    'DATABASE= .....;' // Database name của bạn
    'Trusted_Connection=yes;'
)

def get_available_tables():
    """Truy vấn danh sách bảng từ Database"""
    try:
        conn = pyodbc.connect(conn_str)
        query = "SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE'"
        tables_df = pd.read_sql(query, conn)
        conn.close()
        return tables_df['TABLE_NAME'].tolist()
    except Exception as e:
        print(f"Lỗi kết nối: {e}")
```

```

        return []

def export_multiple_tables_to_hyper(selected_tables):
    """
    Sử dụng biến toàn cục OUTPUT_FILE để xuất dữ liệu
    """
    # Khai báo sử dụng biến toàn cục
    global OUTPUT_FILE

    if not selected_tables:
        print("Không có bảng nào được chọn.")
        return

    conn = pyodbc.connect(conn_str)
    data_dict = {}

    print(f"\n--- Đang bắt đầu quá trình trích xuất ---")

    for table in selected_tables:
        try:
            print(f"Đang đọc: {table}")
            query = f"SELECT * FROM {table}"
            df = pd.read_sql(query, conn)
            data_dict[TableName("Extract", table)] = df
        except Exception as e:
            print(f"Lỗi tại bảng {table}: {e}")

    try:
        # Sử dụng giá trị từ biến toàn cục làm tên file đầu ra
        pantab.frames_to_hyper(data_dict, OUTPUT_FILE)
        print(f"\n[THÀNH CÔNG] Dữ liệu đã được cập nhật vào file: {OUTPUT_FILE}")
    except Exception as e:
        print(f"Lỗi khi ghi file: {e}")
    finally:
        conn.close()

# --- LƯỖNG CHẠY CHƯƠNG TRÌNH ---
if __name__ == "__main__":
    all_tables = get_available_tables()

    if all_tables:
        print("Danh sách bảng hiện có:")
        for index, table in enumerate(all_tables, start=1):
            print(f"{index}. {table}")

        user_choice = input("\nNhập số thứ tự bảng bạn muốn chọn (có thể nhập nhiều số, cách nhau bởi dấu phẩy): ")
        selected_indexes = [item.strip() for item in user_choice.split(',') if
            item.strip().isdigit()]
        selected_list = []

        for idx_str in selected_indexes:
            idx = int(idx_str)
            if 1 <= idx <= len(all_tables):
                selected_list.append(all_tables[idx - 1])
            else:
                print(f"Số {idx} không hợp lệ và sẽ bị bỏ qua.")

        if selected_list:
            export_multiple_tables_to_hyper(selected_list)
        else:
            print("Không có bảng hợp lệ được chọn.")

```


3. Quy trình vận hành và Reload trong Tableau

Lần đầu tiên kết nối:

4. Chạy script Python để tạo file “DuLieu_Tu_MSSQL.hyper”.
5. Mở **Tableau Public**.
6. Tại màn hình Connect, chọn **More...** dưới mục **To a File** và chọn file .hyper vừa tạo.
7. Bạn sẽ thấy tất cả các bảng bạn đã chọn hiện lên trong Data Source. Bạn có thể kéo chúng vào để Join hoặc Relationship bình thường.

Cách Reload dữ liệu (Khi DB có cập nhật):

1. Bạn không cần làm gì trong Tableau. Hãy chạy lại script Python trên.
2. Script sẽ tự động quét SQL Server và ghi đè dữ liệu mới nhất vào chính file .hyper đó.
3. Trong Tableau, bạn chỉ cần vào menu **Data -> [Tên Data Source] -> Refresh**. Dữ liệu sẽ lập tức được cập nhật mới nhất từ file Hyper.